

Constrained Update for Continual Reinforcement Learning

Setup

Tasks $1, \dots, k$ have appeared; k is the current task and $i \in \{1, \dots, k\}$ a generic index. Task i is the MDP

$$M_i = (S, A, P_i, r_i, \rho_i, \gamma), \quad (1)$$

with states S , actions A , transition law $P_i(s' | s, a)$, reward $r_i(s, a)$, initial distribution ρ_i , and discount $\gamma \in (0, 1)$.

The local policy is $\pi_L = \pi_\theta$ and the global policy is $\pi_G = \pi_\phi$, with parameters θ and ϕ . The two are trained in alternation: during a local update, θ is optimized while π_G is held fixed; during a global update, ϕ is optimized while π_L is held fixed. Each local update starts from the global,

$$\theta^{(0)} = \phi. \quad (2)$$

We use uniform task weights for now, $\omega_i = 1/k$.

A length- T trajectory is $\tau = (s_0, a_0, \dots, s_T, a_T, s_{T+1})$, with law under policy π on task i

$$p_i^\pi(\tau) = \rho_i(s_0) \prod_{t=0}^T \pi(a_t | s_t) P_i(s_{t+1} | s_t, a_t). \quad (3)$$

The discounted return and the value of π on task i are

$$G_i(\tau) = \sum_{t=0}^T \gamma^t r_i(s_t, a_t), \quad V_i^\pi = \mathbb{E}_{\tau \sim p_i^\pi} [G_i(\tau)]. \quad (4)$$

We assume access to all previously seen environments. Thus old-task terms are estimated from fresh rollouts in old environments rather than from a stored replay buffer; the method is replay-free in that sense.

The objective and the constraint

We score a policy by the weighted sum of its per-task values, the weights $\omega_i \geq 0$ fixing how much each task counts,

$$J_k(\pi) = \sum_{i=1}^k \omega_i V_i^\pi. \quad (5)$$

For a new policy π_{new} and a reference policy π_{ref} , the total improvement of one policy over another can be split into current-task improvement and past-task improvement:

$$\begin{aligned} J_k(\pi_{\text{new}}) - J_k(\pi_{\text{ref}}) &= \sum_{i=1}^k \omega_i V_i^{\pi_{\text{new}}} - \sum_{i=1}^k \omega_i V_i^{\pi_{\text{ref}}} \\ &= \sum_{i=1}^k \omega_i (V_i^{\pi_{\text{new}}} - V_i^{\pi_{\text{ref}}}) \\ &= \underbrace{\omega_k (V_k^{\pi_{\text{new}}} - V_k^{\pi_{\text{ref}}})}_{\text{current task}} + \underbrace{\sum_{i=1}^{k-1} \omega_i (V_i^{\pi_{\text{new}}} - V_i^{\pi_{\text{ref}}})}_{\text{past tasks}}. \end{aligned} \quad (6)$$

The local model is intentionally left unconstrained so that it can fully adapt to the current task. Past-task preservation is enforced only during the subsequent global update. During the global update, the local policy serves as the reference for the current task, while the global policy is encouraged to retain performance on previously learned tasks.

Local objective

The local model updates π_L with the global held fixed. Each local update is initialized from the current global policy according to (2). Empirically, constraining the local model reduced its ability to sufficiently learn the current task. Therefore, the local model is left unconstrained and is optimized solely to maximize its expected return on the current task. Any forgetting introduced during this stage is addressed during the subsequent global update.

The local objective is

$$J_L(\pi_L) = V_k^{\pi_L}, \quad (7)$$

and the corresponding optimization problem is

$$\max_{\pi_L} J_L(\pi_L), \quad (8)$$

where the optimization is performed only on the current task.

Global objective

The global model updates π_G with the local held fixed. It maximizes its past lead over the local and is kept within tolerance $\epsilon \geq 0$ of the local on the current task. The same one-sided idea is used here: the global model is penalized only when its current-task value is below the local model's current-task value. With the current-task shortfall

$$F_G(\pi_G) = \begin{cases} (V_k^{\pi_L} - V_k^{\pi_G})^2, & \text{if } V_k^{\pi_G} < V_k^{\pi_L}, \\ 0, & \text{if } V_k^{\pi_G} \geq V_k^{\pi_L}, \end{cases} \quad (9)$$

the global problem, its Lagrangian, and its saddle point are

$$\max_{\pi_G} \sum_{i=1}^{k-1} \omega_i (V_i^{\pi_G} - V_i^{\pi_L}) \quad \text{s.t.} \quad F_G(\pi_G) \leq \epsilon, \quad (10)$$

$$\mathcal{L}_G(\pi_G, \mu) = \sum_{i=1}^{k-1} \omega_i (V_i^{\pi_G} - V_i^{\pi_L}) - \mu (F_G(\pi_G) - \epsilon), \quad (11)$$

$$\max_{\pi_G} \min_{\mu \geq 0} \mathcal{L}_G(\pi_G, \mu). \quad (12)$$

During alternation, the local update maximizes performance on the current task without constraints. The subsequent global update consolidates this newly acquired knowledge while preserving performance on previous tasks through constrained optimization.

Notes for experiments. A scalar value constraint may not always imply that the global policy has absorbed the local policy's behavior; this should be checked empirically before adding a distillation term. We should also monitor feasibility: persistent constraint violations or diverging multipliers indicate task conflict or insufficient model capacity. Since the old-task reference is the current global value, cumulative forgetting should be checked in final per-task evaluation.

Local update

Optimizing over the local policy means optimizing over its parameters θ , since $\pi_L = \pi_\theta$. The local objective is unconstrained and is optimized by gradient ascent on $J_L(\theta)$. The update requires the policy gradient $\nabla_\theta V_k^{\pi_\theta}$, which is derived below. Since the derivation applies to an arbitrary task, we first derive $\nabla_\theta V_i^{\pi_\theta}$ for a generic task i .

The chain rule on the logarithm gives $\nabla_\theta \log p_i^{\pi_\theta}(\tau) = \nabla_\theta p_i^{\pi_\theta}(\tau) / p_i^{\pi_\theta}(\tau)$, and multiplying both sides by $p_i^{\pi_\theta}(\tau)$,

$$\nabla_\theta p_i^{\pi_\theta}(\tau) = p_i^{\pi_\theta}(\tau) \nabla_\theta \log p_i^{\pi_\theta}(\tau).$$

With finite S and A the value (4) is a sum over all trajectories, $V_i^{\pi_\theta} = \sum_\tau p_i^{\pi_\theta}(\tau) G_i(\tau)$. Substituting the identity,

$$\begin{aligned} \nabla_\theta V_i^{\pi_\theta} &= \nabla_\theta \sum_\tau p_i^{\pi_\theta}(\tau) G_i(\tau) \\ &= \sum_\tau \nabla_\theta p_i^{\pi_\theta}(\tau) G_i(\tau) \\ &= \sum_\tau p_i^{\pi_\theta}(\tau) \nabla_\theta \log p_i^{\pi_\theta}(\tau) G_i(\tau) \\ &= \mathbb{E}_{\tau \sim p_i^{\pi_\theta}} [G_i(\tau) \nabla_\theta \log p_i^{\pi_\theta}(\tau)]. \end{aligned} \quad (13)$$

In the trajectory law (3) only the policy factors depend on θ , so ρ_i and P_i drop from the log-gradient,

$$\begin{aligned}
\nabla_\theta \log p_i^{\pi_\theta}(\tau) &= \nabla_\theta \left[\log \rho_i(s_0) + \sum_{t=0}^T \log \pi_\theta(a_t | s_t) + \sum_{t=0}^T \log P_i(s_{t+1} | s_t, a_t) \right] \\
&= \underbrace{\nabla_\theta \log \rho_i(s_0)}_0 + \sum_{t=0}^T \nabla_\theta \log \pi_\theta(a_t | s_t) + \underbrace{\sum_{t=0}^T \nabla_\theta \log P_i(s_{t+1} | s_t, a_t)}_0 \\
&= \sum_{t=0}^T \nabla_\theta \log \pi_\theta(a_t | s_t).
\end{aligned} \tag{14}$$

Substituting (14) into (13) and factoring out the return, which does not carry the index t , gives the per-task policy gradient

$$\nabla_\theta V_i^{\pi_\theta} = \mathbb{E}_{\tau \sim p_i^{\pi_\theta}} \left[G_i(\tau) \sum_{t=0}^T \nabla_\theta \log \pi_\theta(a_t | s_t) \right]. \tag{15}$$

During the local update, the optimization objective is

$$J_L(\theta) = V_k^{\pi_\theta}, \tag{16}$$

Differentiating the objective with respect to θ gives

$$\nabla_\theta J_L(\theta) = \nabla_\theta V_k^{\pi_\theta}. \tag{17}$$

Substituting the policy gradient from (15),

$$\nabla_\theta J_L(\theta) = \mathbb{E}_{\tau \sim p_k^{\pi_\theta}} \left[G_k(\tau) \sum_{t=0}^T \nabla_\theta \log \pi_\theta(a_t | s_t) \right]. \tag{18}$$

Using the Monte Carlo estimator,

$$\widehat{V}_k^{\pi_\theta} = \frac{1}{N} \sum_{n=1}^N G_k(\tau^{(n)}), \tag{19}$$

and

$$\overline{\nabla_\theta V_k^{\pi_\theta}} = \frac{1}{N} \sum_{n=1}^N G_k(\tau^{(n)}) \sum_{t=0}^T \nabla_\theta \log \pi_\theta(a_t^{(n)} | s_t^{(n)}), \tag{20}$$

the local parameters are updated by gradient ascent,

$$\begin{aligned}
\theta^{(m+1)} &= \theta^{(m)} + \alpha \nabla_\theta J_L(\theta) \\
&= \theta^{(m)} + \alpha \overline{\nabla_\theta V_k^{\pi_\theta}}.
\end{aligned} \tag{21}$$

Global update

Optimizing over the global policy means optimizing over its parameters ϕ , since $\pi_G = \pi_\phi$. A saddle point of (12) is approached by ascending $\mathcal{L}_G$ in ϕ and descending it in μ . Both directions need $\nabla_\phi \mathcal{L}_G$, built from the per-task gradients $\nabla_\phi V_i^{\pi_\phi}$. The derivation (13)–(15) used nothing about θ beyond the fact that only the policy factors of the trajectory law (3) carry the parameter, so it holds verbatim with ϕ in place of θ ,

$$\nabla_\phi V_i^{\pi_\phi} = \mathbb{E}_{\tau \sim p_i^{\pi_\phi}} \left[G_i(\tau) \sum_{t=0}^T \nabla_\phi \log \pi_\phi(a_t | s_t) \right]. \tag{22}$$

During the global update, the local policy is held fixed. Therefore,

$$\nabla_\phi V_i^{\pi_L} = 0, \quad \nabla_\phi V_k^{\pi_L} = 0, \quad \nabla_\phi \epsilon = 0.$$

For the squared shortfall in (9),

$$\begin{aligned}
\nabla_\phi F_G(\pi_\phi) &= \nabla_\phi (V_k^{\pi_L} - V_k^{\pi_\phi})^2 \\
&= 2 (V_k^{\pi_L} - V_k^{\pi_\phi}) \nabla_\phi (V_k^{\pi_L} - V_k^{\pi_\phi}) \\
&= 2 (V_k^{\pi_L} - V_k^{\pi_\phi}) (0 - \nabla_\phi V_k^{\pi_\phi}) \\
&= -2 (V_k^{\pi_L} - V_k^{\pi_\phi}) \nabla_\phi V_k^{\pi_\phi},
\end{aligned} \tag{23}$$

$$\nabla_{\phi} F_G(\pi_{\phi}) = \begin{cases} -2(V_k^{\pi_L} - V_k^{\pi_{\phi}}) \nabla_{\phi} V_k^{\pi_{\phi}}, & \text{if } V_k^{\pi_{\phi}} < V_k^{\pi_L}, \\ 0, & \text{if } V_k^{\pi_{\phi}} \geq V_k^{\pi_L}. \end{cases} \quad (24)$$

Differentiating the Lagrangian (11) with respect to ϕ ,

$$\begin{aligned} \nabla_{\phi} \mathcal{L}_G &= \nabla_{\phi} \left[\sum_{i=1}^{k-1} \omega_i (V_i^{\pi_{\phi}} - V_i^{\pi_L}) - \mu (F_G - \epsilon) \right] \\ &= \sum_{i=1}^{k-1} \omega_i (\nabla_{\phi} V_i^{\pi_{\phi}} - \nabla_{\phi} V_i^{\pi_L}) - \mu (\nabla_{\phi} F_G - \nabla_{\phi} \epsilon) \\ &= \sum_{i=1}^{k-1} \omega_i \nabla_{\phi} V_i^{\pi_{\phi}} - \mu \nabla_{\phi} F_G \\ &= \sum_{i=1}^{k-1} \omega_i \nabla_{\phi} V_i^{\pi_{\phi}} - \mu [-2(V_k^{\pi_L} - V_k^{\pi_{\phi}}) \nabla_{\phi} V_k^{\pi_{\phi}}] \\ &= \sum_{i=1}^{k-1} \omega_i \nabla_{\phi} V_i^{\pi_{\phi}} + 2\mu (V_k^{\pi_L} - V_k^{\pi_{\phi}}) \nabla_{\phi} V_k^{\pi_{\phi}}. \end{aligned} \quad (25)$$

since both the local policy and the tolerance are fixed during the global update.

$$\nabla_{\phi} \mathcal{L}_G(\pi_{\phi}, \mu) = \sum_{i=1}^{k-1} \omega_i \nabla_{\phi} V_i^{\pi_{\phi}} + \mu \begin{cases} 2(V_k^{\pi_L} - V_k^{\pi_{\phi}}) \nabla_{\phi} V_k^{\pi_{\phi}}, & \text{if } V_k^{\pi_{\phi}} < V_k^{\pi_L}, \\ 0, & \text{if } V_k^{\pi_{\phi}} \geq V_k^{\pi_L}, \end{cases} \quad (26)$$

each per-task gradient given by (22).

Drawing N trajectories $\tau^{(1)}, \dots, \tau^{(N)}$ on task i under π_{ϕ} , the per-task value and gradient are estimated by

$$\hat{V}_i^{\pi_{\phi}} = \frac{1}{N} \sum_{n=1}^N G_i(\tau^{(n)}), \quad (27)$$

and

$$\overline{\nabla_{\phi} V_i^{\pi_{\phi}}} = \frac{1}{N} \sum_{n=1}^N G_i(\tau^{(n)}) \sum_{t=0}^T \nabla_{\phi} \log \pi_{\phi}(a_t^{(n)} | s_t^{(n)}). \quad (28)$$

During the same global update, $\hat{V}_k^{\pi_L}$ denotes the rollout estimate of the held-fixed local value on the current task. The global primal update is

$$\phi^{(m+1)} = \phi^{(m)} + \beta \left[\sum_{i=1}^{k-1} \omega_i \overline{\nabla_{\phi} V_i^{\pi_{\phi}}} + \mu^{(m)} \begin{cases} 2(\hat{V}_k^{\pi_L} - \hat{V}_k^{\pi_{\phi}}) \overline{\nabla_{\phi} V_k^{\pi_{\phi}}}, & \text{if } \hat{V}_k^{\pi_{\phi}} < \hat{V}_k^{\pi_L}, \\ 0, & \text{if } \hat{V}_k^{\pi_{\phi}} \geq \hat{V}_k^{\pi_L}, \end{cases} \right]. \quad (29)$$

The multiplier is obtained by minimizing the Lagrangian with respect to μ . Since

$$\frac{\partial \mathcal{L}_G}{\partial \mu} = -(F_G - \epsilon), \quad (30)$$

gradient descent on μ gives

$$\begin{aligned} \mu^{(m+1)} &= \mu^{(m)} - \eta_{\mu} \frac{\partial \mathcal{L}_G}{\partial \mu} \\ &= \mu^{(m)} + \eta_{\mu} (F_G - \epsilon). \end{aligned} \quad (31)$$

Finally, because the Lagrange multiplier must satisfy $\mu \geq 0$, it is projected onto the nonnegative orthant,

$$\mu^{(m+1)} = \max \{0, \mu^{(m)} + \eta_{\mu} (\hat{F}_G^{(m)} - \epsilon)\}, \quad (32)$$

where

$$\hat{F}_G^{(m)} = \begin{cases} (\hat{V}_k^{\pi_L} - \hat{V}_k^{\pi_{\phi}^{(m)}})^2, & \text{if } \hat{V}_k^{\pi_{\phi}^{(m)}} < \hat{V}_k^{\pi_L}, \\ 0, & \text{if } \hat{V}_k^{\pi_{\phi}^{(m)}} \geq \hat{V}_k^{\pi_L}. \end{cases} \quad (33)$$

After the global phase for task k , the new task becomes part of the past-task set for the next task.